

Hypo-Stage: Tool-Supported Architectural Hypothesis Engineering in Professional Software Development Teams — A Case Study

José Gonçalves Lima Neto

11 June 2026

Master's Thesis Defense

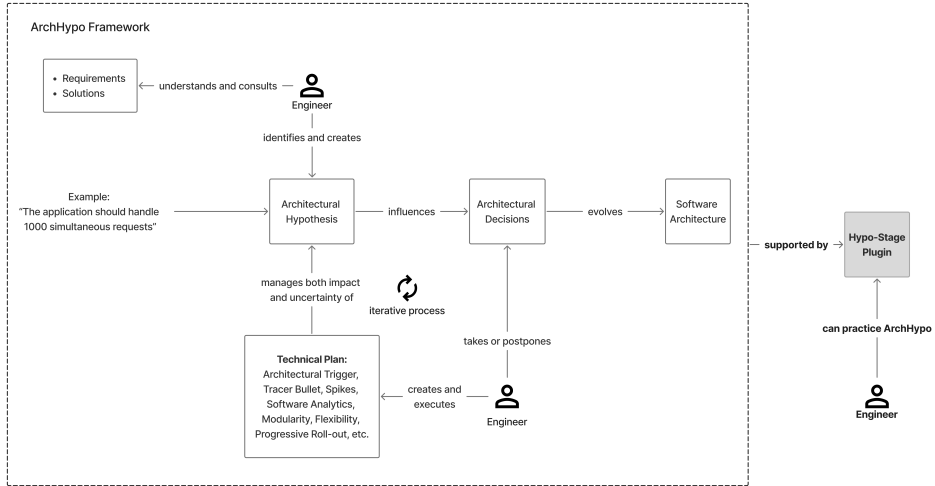
Advisor: Prof. Dr. Paulo Roberto Miranda Meirelles

Graduate Program in Computer Science

IME – University of São Paulo



Context



Problem & research questions

The Problem

- Deriving architectures still depends on architects' **tacit knowledge** and lacks **tool support** (Souza et al., 2019).
- ArchHypo structures uncertainty, but teams perceive it as **hard to learn** and missed guidance via supporting tools (Silva, Melegati, Silveira, et al., 2025; Silva, Melegati, Wang, et al., 2024).
- Hypo-Stage gives the initial tool support, but **impact on real teams** and refinements remained **unknown** (Corrêa, 2025).

Research questions

RQ1. How do teams *use* Hypo-Stage and ArchHypo on a scaled product?

RQ2. What *uncertainties* and recurring *patterns* do they record?

RQ3. What *benefits and challenges* do team members perceive?

RQ4. Which *refinements* to the tool emerge, and how do they address the challenges?

Study setup

Environment

Single organization

Scaled product

Backstage IDP

Teams

Product team

12 engineers

Platform team

4 engineers

Timeline

5 sprints
(≈ 9 weeks)

Sprint 1

Sprint 2

Sprint 3

Sprint 4

Sprint 5

Artifact analysis
(13 hypotheses)

Data sources

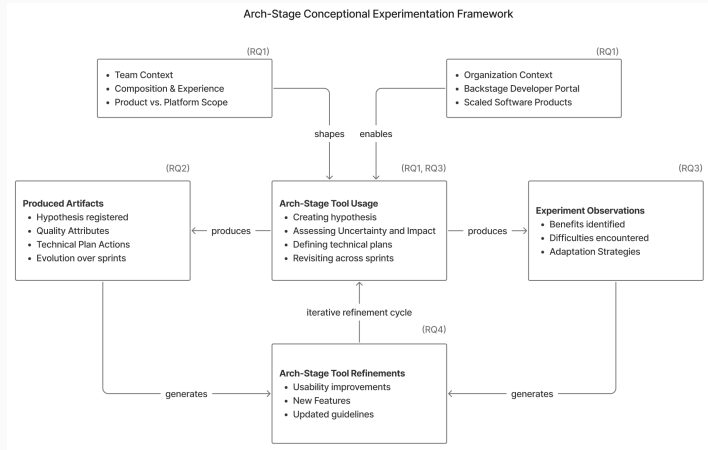
triangulated

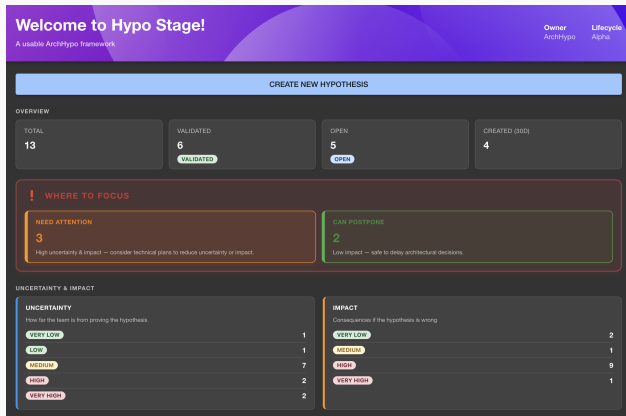
Participant
observation

Active-listening
sessions

Research Method

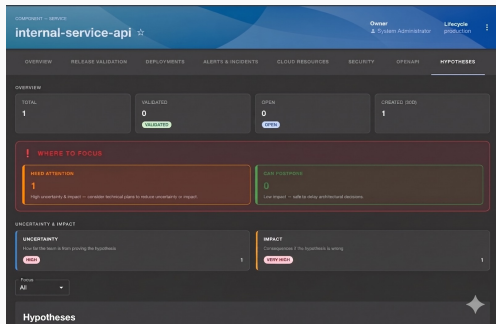
Grounded in the **flexible design** approach to real-world research (Robson and McCartan, 2016).





13 hypotheses, two teams — adoption was **senior-led** and **event-anchored**:

- 12 of 13 authored by senior/staff engineers
- Seniors used it for **decision support**; junior/mid uncertainties resolved within a sprint, so went unregistered
- Registration clustered around protected technical sessions
- Backstage integration cut **context-switching** cost



13 hypotheses (6 Product, 7 Platform), surfaced per service in the portal: 8 requirements- / 5 solution-sourced; 9 of 13 carry high impact. Dominant quality attributes: Reliability & Auditability.

Five emerging hypothesis patterns

First empirically grounded answer to *what shapes* hypotheses take in practice.

- **Hypothesis Cluster** — one platform concern → linked hypotheses
- **Regulatory Trigger** — compliance obligation as tracked uncertainty
- **Context-Agnostic Restructuring** — whether/how to decouple boundaries
- **Load-Validated** — proven under load, implementation still open
- **Shared Resource Governance** — ownership rules for shared infrastructure

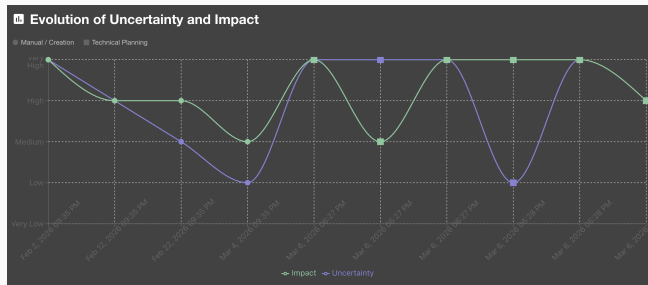
Benefits

- Externalizes tacit knowledge — a written hypothesis becomes shareable beyond the original discussion
- Supports prioritization — “**Can Postpone**” justifies deliberate deferral, not neglect
- Collective memory — records open questions before a decision, where ADRs cannot

Challenges

- Falsifiable phrasing — the **negative formulation** is non-intuitive and easy to skip under pressure
- Registration time cost — short-lived uncertainties go **unregistered**
- No team ceremonies — usage stayed **individual**, never a shared review ritual

9 refinement categories — including prioritization, the evolution chart, and form usability — each traced to an observed friction point.



Evolution chart: uncertainty & impact tracked across successive assessments.

The "Create New Hypothesis" form includes fields for "Entity References", "Hypothesis Statement" (with a 5000 character limit), and "Source Type". It also features two rating scales: "Uncertainty Level" (How far is the team from validating/falsifying this hypothesis?) and "Impact Level" (What is the potential impact on the system if the hypothesis proves false?). Both scales range from 1 to 5, with "Not rated" as an option. A "Quality Attributes" section at the bottom allows users to select one or more attributes, with a note to click the field to open a list of options shown in parentheses.

Inline formulation guidance in the creation form lowers the falsifiability barrier.

Implications & lessons learned

Finding

Even with tool support, hypothesis work stayed **senior-anchored** (12 of 13).

Implication

A tool removes **friction** but cannot supply the **organizational conditions** for broad participation.

Lesson

Ship hypothesis support **inside** the developer portal — *extend* the workflow, don't add a new one.

Takeaway: distributed architectural work needs lightweight structure that preserves team autonomy while protecting shared decision quality.

Future work & closing

Four strategic directions:

Replicate
across orgs

Longitudinal
lifecycle study

Integrate with
ADR tools

Broaden
participation

Takeaway

Tool-mediated ArchHypo succeeds at **integration and traceability**; the remaining challenge is **organizational and pedagogical**.

Limitations: single organization · 13 hypotheses · ≈ 9 weeks · researcher as participant.

Emerging patterns, not consolidated findings.

Thank you!

Questions & discussion are welcome.

José Gonçalves Lima Neto

Advisor: Prof. Dr. Paulo Roberto Miranda Meirelles

Graduate Program in Computer Science — IME-USP

Master's Thesis Defense · 11 June 2026

